



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**RESOURCEHUB: A SECURE CAMPUS RESOURCE SHARING AND EXCHANGE
PLATFORM WITH QR-BASED HANDOVER VERIFICATION**

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the course of
CSA4305 – INTERNET PROGRAMMING

to the award of the degree of
BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

Submitted By
Ebin Antony P (192373011),

Under the Supervision of
DR. D. Sudhagar
SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

SEPTEMBER 2026



**SIMATS ENGINEERING,
SAVEETHA INSTITUTE OF MEDICAL AND
TECHNICAL SCIENCE, CHENNAI- 602105**



BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**Campus Resource Exchange Network(CREN): An Intelligent Academic Marketplace for Resource Sharing**” has been carried out by Ebin Antony P (192373011), under the supervision of **DR. D. Sudhagar** and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Computer Science Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr Marie Claude .N
Professor
Department of Neural Networks
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr.D.Sudhagar
Professor
Quantum Intelligence
SIMATS Engineering,
SIMATS, Chennai – 602105.

Submitted for the Project work Viva-Voce held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, Ebin Antony P (192373011) of the CSE Department, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “**Campus Resource Exchange Network (CREN): An Intelligent Academic Marketplace for Resource Sharing**” is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:

Date:

Signature of Student

Ebin Antony P (192373011)

SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCE

CHENNAI- 602105

Individual Contribution Statement

Student Name / Register No.	Module / Responsibility	Design & Development	Testing & Analysis	Report Contribution	Contribution
Meghanadh A (192372281)	Module 1 – User Authentication	Authentication & user management	Authentication testing	Module 1 documentation	20%
Aaron Samuel S (192411022)	Module 2 – Resource Management	Resource CRUD & categories	Resource testing	Module 2 documentation	20%
Deepak V (192411021)	Module 3 – Exchange Management	Exchange requests & transactions	Transaction testing	Module 3 documentation	20%
Y. Abhay (192411219)	Module 4 – QR Verification & Trust System	QR verification & trust system	QR & trust testing	Module 4 documentation	20%
Ebin Antony P (192373011)	Module 5 – Supporting Features	Notifications, wishlist & reports	Feature testing	Module 5 documentation	20%
Total					100%

ABSTRACT

ResourceHub is a web-based platform designed to provide a structured and secure environment for students to share, request, reserve, and exchange useful resources within a campus community. In educational institutions, students frequently possess books, electronic devices, laboratory equipment, study materials, and other resources that may be temporarily unused but could be useful to other students. However, conventional resource-sharing methods often depend on informal communication, manual coordination, and unstructured exchanges, which can result in difficulty in discovering resources, tracking requests, and verifying successful handovers.

A major feature of the proposed system is the **QR-based handover verification mechanism**. After a resource request is accepted, the resource owner can generate a temporary handover QR code. The requester scans the QR code to verify the physical exchange. The verification process includes security measures such as token expiry, protection against unauthorized users, prevention of self-scanning, and prevention of repeated verification. Once the handover is successfully verified, the transaction is completed and the resource status is updated accordingly.

The system demonstrates how web technologies, database management, authentication, transaction management, and QR-based verification can be integrated to create a reliable campus resource-sharing platform. The project provides a practical solution for improving resource utilization, reducing unnecessary purchases, and encouraging collaborative sharing among students. Future enhancements can include trust scores, ratings and reviews, notifications, advanced search, analytics, and integration with institutional systems.

Keywords: Resource Sharing, Campus Platform, Resource Exchange, QR Verification, Handover Verification, Reservation Management, Secure Transactions, Student Collaboration.

ACKNOWLEDGEMENT

We wish to express our sincere thanks. Behind every achievement lies an unfathomable sea of gratitude to those who actuated it; without them, it would never have existed. We sincerely thank our respected founder and Chancellor **Dr. N.M. Veeraiyan**, Saveetha Institute of Medical and Technical Science, for his blessings and for being a source of inspiration. We sincerely thank our Pro-Chancellor, **Dr. Deepak Nallaswamy Veeraiyan**, SIMATS, for his visionary thoughts and support. We sincerely thank our Vice-Chancellor **Dr. S. Suresh Kumar**, SIMATS, for your moral support throughout the project. We are indebted to extend our gratitude to our Director **Dr. Ramya Deepak**, SIMATS Engineering, for facilitating all the facilities and extended support to gain valuable education and learning experience.

We give special thanks to our Principal **Dr. B Ramesh**, SIMATS Engineering and **Dr S Srinivasan**, Vice Principal SIMATS Engineering, for allowing us to use institute facilities extensively to complete this capstone project effectively. I Express my sincere thanks to our guide, **Dr. D. Sudhagar**, for continuous help over the period and creative ideas for this capstone project for his inspiring guidance, personal involvement and constant encouragement during this work.

We are grateful to the Project Coordinators, Review Panel External and Internal Members and the entire faculty for their constructive criticisms and valuable suggestions, which have been a rich source of improvements in the quality of this work. We want to extend our warmest thanks to all faculty members, lab technicians, parents, and friends for their support.

Sincerely,

**Deepak V (192411021),
Aaron Samuel S (192411022),
Y. Abhay (192411219),
Ebin Antony P (192373011),
Meghanadh A (192372281).**

TABLE OF CONTENTS

S.NO	CONTENTS	PG.NO
1.	Introduction	9-10
	1.1 Background Information	9
	1.2 Project Objectives	9
	1.3 Significance	9
	1.4 Scope	10
	1.5 Methodology Overview	10
2.	Problem Identification and Analysis	11-12
	2.1 Description of the problem	11
	2.2 Evidence of the problem	11
	2.3 Stakeholders	11
	2.4 Supporting Data/Research	12
3.	Solution Design and Implementation	13-15
	3.1 Development & Design process	13
	3.2 Tools & Technologies used	13
	3.3 Solution Overview	14
	3.4 Engineering Standards Applied	14
	3.5 Solution Justification	15
4.	Results and Recommendations	16-18

	4.1 Screenshots	16
	4.2 Evaluation of Results	17
	4.3 Challenges Faced	17
	4.4 Proposed Improvements	18
	4.5 Future Recommendations	18
5.	Reflection on Learning and Personal Development	19-21
	5.1 Key Learning Outcomes	19
	5.2 Challenges Encountered and Overcome	19
	5.3 Applications of Engineering Standards	20
	5.4 Industry Insights	20
	5.5 Conclusion of Personal Development	21
6.	Conclusion	22
7.	References	23
8.	Appendices	24-29

LIST OF FIGURES

FIG NO	TITLE	PAGE NO
3.1	ResourceHub Development Architecture	11
4.1	<i>User Registration and Login Interface</i>	11
4.2	ResourceHub Dashboard	12
4.3	Resource Listing and Request Interface	13
4.4	Request Management Interface	16

LIST OF TABLES

TABLE NO	Table Name	PAGE NO
3.2	Tools & Technologies Used	17
4.2	Evaluation of Results	22
4.3	API Endpoints	33
4.4	Testing Checklist	33

LIST OF ABBREVIATIONS / SYMBOLS

Symbol / Abbreviation	Description	Unit
API	Application Programming Interface	—
CRUD	Create, Read, Update, Delete	—
DB	Database	—
JWT	JSON Web Token	—
QR	Quick Response	—
UI	User Interface	—
UX	User Experience	—
SQL	Structured Query Language	—
HTTP	Hypertext Transfer Protocol	—
URL	Uniform Resource Locator	—
%	Percentage	%
ID	Identifier	—
OTP	One-Time Password	—
Vite	Frontend Build Tool	—
—	Not Applicable	—

CHAPTER 1

INTRODUCTION

1.1 Background Information

In modern educational institutions, students frequently require access to books, study materials, electronic devices, laboratory equipment, project components, and other useful resources. At the same time, many students already possess resources that remain unused for long periods. A structured resource-sharing system can connect students who have available resources with students who need them, thereby improving resource utilization and encouraging collaborative practices within the campus community.

Traditional campus resource sharing is commonly performed through informal communication methods such as personal conversations, messaging groups, social media groups, or word-of-mouth communication. Although these methods are convenient for small-scale exchanges, they do not provide a centralized mechanism for discovering available resources, submitting requests, tracking reservations, or maintaining transaction status. As the number of users and resources increases, managing such exchanges becomes more difficult and less transparent.

Another important challenge is the verification of the actual physical handover. A digital request or reservation alone does not guarantee that the resource has been successfully exchanged between the two participants. Without a reliable verification mechanism, transactions may remain incomplete, resources may be incorrectly marked as available, or disputes may occur between users.

To address these challenges, **ResourceHub** is proposed as a secure campus resource-sharing and exchange platform. The system provides users with a centralized environment for listing resources, browsing available resources, submitting requests, managing reservations, and completing exchanges. The platform also introduces QR-based handover verification, allowing the physical exchange to be securely confirmed before a transaction is marked as completed.

The proposed system combines resource management, request processing, reservation control, user authentication, and QR-based verification into a single platform. This approach improves transparency in campus resource exchange while promoting reuse, collaboration, and efficient utilization of existing resources.

1.2 Project Objectives

The main objectives of the ResourceHub project are:

- To develop a centralized platform for sharing and exchanging resources within a campus community.
- To allow resource owners to list and manage resources that are available for sharing or exchange.
- To enable students to discover resources and submit requests through a structured digital workflow.
- To implement reservation and transaction status management to prevent conflicting resource requests.
- To provide secure QR-based handover verification for confirming successful physical exchanges.
- To improve transparency, reliability, and accountability in campus resource-sharing activities.

1.3 Significance

ResourceHub provides a practical solution for improving the utilization of resources within educational institutions. Instead of allowing useful resources to remain unused, students can make them available to others who may require them. This can reduce unnecessary purchases and encourage a culture of sharing and collaboration.

The platform is also significant from a software engineering perspective because it demonstrates the integration of multiple application components including user authentication, resource management, request processing, database operations, reservation handling, and QR-based transaction verification.

The QR handover mechanism further improves the reliability of the platform by connecting the digital transaction with the actual physical exchange. By requiring verification before completing a transaction, the system provides an additional layer of accountability between resource owners and requesters.

1.4 Scope

The scope of the ResourceHub system includes:

- User registration and authentication.
- Creation and management of resource listings.
- Browsing and searching available resources.
- Submission and management of resource requests.
- Acceptance and rejection of requests by resource owners.
- Resource reservation and transaction status management.
- QR-based handover verification.
- Temporary QR/token expiry mechanisms.
- Prevention of unauthorized and repeated handover verification.
- Updating resource and transaction status after successful exchange.

The system is primarily designed for campus-level resource sharing and can be extended in the future to support larger communities and additional resource categories.

1.5 Methodology Overview

The development of ResourceHub follows a systematic software development approach. Initially, the requirements of campus resource sharing are identified, including resource listing, request management, reservation, and secure exchange verification. These requirements are then translated into system modules and database structures.

During the design stage, the major components of the system are organized into user authentication, resource management, request processing, reservation management, and handover verification modules. Appropriate interfaces are designed to provide separate workflows for resource owners and requesters.

The implementation stage focuses on developing the individual modules and integrating them into a unified platform. Database operations are implemented to maintain users, resources, requests,

reservations, and transaction states. Security mechanisms are incorporated into the QR handover process to prevent unauthorized or repeated verification.

Testing is then performed using different user scenarios and transaction states. The system is evaluated for functional correctness, secure handover verification, status transitions, and prevention of invalid operations. The final results are analyzed to determine whether the developed platform successfully addresses the identified resource-sharing challenges.

Future enhancements can include rating and review mechanisms, trust scores, notifications, improved search capabilities, analytics, and integration with institutional systems.

CHAPTER 2

PROBLEM IDENTIFICATION AND ANALYSIS

2.1 Description of the Problem

Students in educational institutions regularly require resources such as textbooks, reference materials, electronic devices, laboratory equipment, project components, and other academic items. However, there is often no dedicated centralized platform through which students can easily identify resources available within their campus community. As a result, resource sharing is commonly handled through informal communication channels such as messaging groups, personal contacts, or verbal communication.

These conventional approaches create several difficulties. Students who need a particular resource may not know which other student has it available, while resource owners may find it difficult to reach potential users. The absence of structured resource listings also makes it difficult to determine whether a resource is currently available, reserved, or already exchanged.

Another significant problem is the lack of a systematic request and reservation mechanism. Multiple students may request the same resource, resulting in confusion regarding ownership and availability. Without proper transaction states, it becomes difficult to track whether a request is pending, accepted, reserved, or completed.

The physical handover of a resource presents an additional challenge. A digital request acceptance does not necessarily confirm that the actual resource has been transferred between the two users. Without a reliable verification mechanism, transactions can remain incomplete or resource status can be incorrectly updated.

ResourceHub addresses these problems by providing a centralized digital platform for resource discovery, request management, reservation, and exchange. The system introduces QR-based handover verification to confirm the physical exchange and ensure that a transaction is completed only after successful verification.

2.2 Evidence of the Problem

The limitations of informal resource-sharing methods become more noticeable as the number of students and available resources increases. Communication through multiple messaging groups or personal conversations can make it difficult to locate specific resources and track their availability.

Students may also spend additional time searching for resources that are already available within the campus community. At the same time, unused resources may remain with their owners because there is no organized mechanism for making other students aware of their availability.

The absence of structured transaction management can further lead to problems such as duplicate requests, unclear reservation status, and difficulty determining whether an exchange has actually taken place. These issues demonstrate the need for a centralized platform capable of maintaining resource and transaction information in an organized manner.

The introduction of QR-based verification provides a practical approach to connecting the digital transaction with the physical handover. Instead of relying only on request acceptance, the system can verify the exchange before changing the transaction to its completed state. This improves transparency and provides greater confidence in the resource-sharing process.

2.3 Stakeholders

The major stakeholders involved in the ResourceHub system include:

- **Resource Owners:** Students who list books, devices, academic materials, equipment, or other resources that they are willing to share or exchange. They can manage their listings and respond to incoming requests.
- **Resource Requesters:** Students who search for available resources and submit requests based on their academic or personal requirements. They participate in the reservation and handover process.
- **Campus Community:** The wider student community benefits from improved resource utilization, easier resource discovery, and greater opportunities for sharing and reuse.
- **System Administrators:** Administrators can be responsible for monitoring the platform, managing users and resources, and ensuring that the system operates correctly.

- **Educational Institutions:** Institutions can benefit from improved collaboration among students, increased reuse of resources, and the availability of a structured digital platform for campus-level resource exchange.

2.4 Supporting Data/Research

Research and existing digital sharing practices indicate that centralized platforms can improve the discoverability and utilization of resources by connecting individuals who possess unused resources with those who require them.

The concept of collaborative consumption and peer-to-peer sharing demonstrates the value of allowing users to share resources rather than independently purchasing or maintaining duplicate items. A campus environment is particularly suitable for such a model because students often have similar academic requirements and operate within a shared community.

Digital transaction systems also demonstrate the importance of maintaining clear states throughout a transaction lifecycle. In ResourceHub, states such as **pending, accepted, reserved, verified, and completed** provide a structured representation of the exchange process.

Security is particularly important when digital systems are used to coordinate physical transactions. The QR-based handover mechanism provides a controlled verification step between the resource owner and requester. Temporary tokens, expiry mechanisms, authorization checks, self-scan prevention, and protection against repeated scans help reduce the possibility of invalid handover verification.

The supporting analysis therefore establishes the need for a platform that combines resource discovery, request management, reservation control, and secure physical handover verification. ResourceHub integrates these requirements into a single campus-oriented system.

CHAPTER 3

SOLUTION DESIGN AND IMPLEMENTATION

3.1 Development and Design Process

The ResourceHub platform was developed using a modular and systematic software development approach. The primary objective was to create a secure campus resource-sharing system that enables students to list resources, submit requests, manage reservations, and complete exchanges through a verified handover process. The development process was organized into different stages to ensure that each functional and security requirement was properly addressed.

The major stages followed during development include:

- **Requirement Analysis** – The requirements of students, resource owners, requesters, and administrators were identified. The major requirements included user authentication, resource listing, request management, reservation handling, and secure handover verification.
- **System Design** – The overall architecture of ResourceHub was designed by dividing the application into functional modules. Database entities and relationships were planned to maintain users, resources, requests, reservations, and transaction information.
- **Interface Development** – User interfaces were developed to provide simple and separate workflows for resource owners and requesters. Screens were designed for authentication, resource browsing, resource management, request handling, and handover verification.
- **Backend Development** – Server-side functionality was implemented to process user requests, manage resource availability, update transaction states, and communicate with the database.
- **QR Handover Implementation** – A secure QR-based verification mechanism was implemented to connect the accepted digital transaction with the actual physical handover. Temporary verification tokens are generated for the resource owner and can be scanned by the authorized requester.
- **Testing and Validation** – The system was tested using different transaction scenarios, including valid requests, unauthorized access attempts, expired QR tokens, self-scanning attempts, and repeated verification attempts.

- **Integration and Evaluation** – All modules were integrated and evaluated as a complete system to verify that resource status and transaction status are updated correctly throughout the exchange lifecycle.

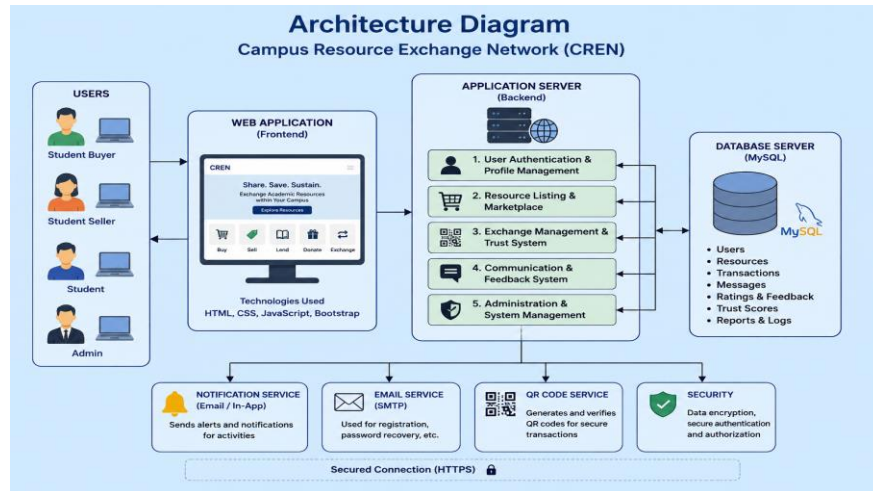


Fig 3.1 ResourceHub Development Architecture

3.2 Tools and Technologies Used

The following tools and technologies were used for the development of the ResourceHub platform:

- **Frontend Technologies:** Used for developing the user interface and interactive components of the application.
- **Backend Technologies:** Used for implementing business logic, authentication, resource management, request processing, and transaction operations.
- **Database:** Used for storing user information, resource details, requests, reservations, and transaction records.
- **QR Code Technology:** Used to generate and scan temporary handover verification codes.
- **Authentication Mechanism:** Used to ensure that only authorized users can access protected resources and transaction operations.
- **Development Environment:** Used for writing, testing, debugging, and integrating the different components of the application.
- **Version Control:** Used to maintain project source code and track development changes.
- **Browser:** Used for accessing and testing the web-based ResourceHub application.

The technologies were selected based on their suitability for developing a modular web application and their ability to support database operations, authentication, secure transactions, and QR-based verification.

Note: Once you give me your exact current technology stack (for example, React + Node.js + Express + MongoDB, or another stack), I can replace these generic technology names with the exact names in your implementation.

3.3 Solution Overview

ResourceHub provides a centralized platform through which students can participate in campus resource sharing and exchange. The system consists of several interconnected modules that manage the complete lifecycle of a resource transaction.

The major functionalities of the proposed system are:

- **User Authentication Module** – Provides registration and login functionality and controls access to protected system operations.
- **Resource Management Module** – Allows resource owners to create, view, update, and manage their resource listings. Resource information can be maintained together with its current availability status.
- **Resource Discovery Module** – Allows requesters to browse available resources and identify items that match their requirements.
- **Request Management Module** – Enables requesters to submit requests for resources and allows owners to accept or reject incoming requests.
- **Reservation and Transaction Module** – Maintains the state of a resource during the request and exchange process. This helps prevent conflicting transactions and provides a clear record of the exchange lifecycle.
- **QR Handover Verification Module** – After an owner accepts a request, a temporary QR verification token can be generated. The requester scans the QR code to confirm the physical handover.
- **Security Validation Module** – Validates the identity and authorization of the users involved in the handover. The system also prevents invalid operations such as self-scanning, unauthorized scanning, expired-token usage, and repeated verification.

- **Transaction Completion Module** – After successful QR verification, the system updates the transaction and resource states to indicate that the exchange has been completed.

Fig 3.2 Overall ResourceHub Workflow

3.4 Engineering Standards Applied

The development of ResourceHub considered software engineering principles and best practices to improve system reliability, maintainability, security, and usability.

The following practices were applied:

- **Modular Design** – The application was divided into independent functional modules such as authentication, resource management, request processing, and handover verification.
- **Secure Authentication and Authorization** – Protected operations are restricted to authenticated and authorized users to prevent unauthorized access to resources and transactions.
- **Input Validation** – User inputs and transaction data are validated before processing to reduce invalid operations and improve system reliability.
- **Database Integrity** – Appropriate database operations and transaction handling are used to maintain consistent resource and request states.
- **Secure Token Handling** – Handover verification uses temporary tokens with expiry controls rather than relying on permanent verification codes.
- **Error Handling** – Invalid requests, expired verification tokens, unauthorized attempts, and other exceptional conditions are handled appropriately.
- **Software Testing Practices** – Functional and security-related test scenarios were performed to verify the behavior of individual modules and the complete transaction workflow.
- **Maintainable Code Structure** – The application was organized into logical components to make future maintenance, debugging, and feature enhancement easier.

These practices support the development of a reliable campus resource-sharing application while maintaining focus on security and transaction correctness.

3.5 Solution Justification

The proposed ResourceHub platform addresses the limitations identified in traditional campus resource-sharing methods by providing a centralized and structured digital environment. Instead of depending on informal communication, students can use the platform to discover resources, submit requests, and track the progress of their transactions.

The resource management and reservation mechanisms provide better visibility into resource availability and reduce confusion caused by multiple simultaneous requests. Clear transaction states allow users and the system to determine whether a request is pending, accepted, reserved, verified, or completed.

The QR-based handover mechanism provides an additional layer of reliability by ensuring that the physical exchange is verified before the transaction is marked as completed. Security controls such as temporary token expiry, authorization checks, self-scan prevention, and protection against repeated verification help reduce the possibility of invalid handover operations.

Therefore, the proposed solution provides a practical approach to campus resource sharing by combining **resource management, request processing, reservation control, and secure QR-based handover verification** into a single system.

CHAPTER 4

RESULTS AND RECOMMENDATIONS

4.1 Screenshots

The developed ResourceHub platform was tested through different user workflows to verify the functionality of resource sharing, request management, reservation, and QR-based handover verification. The following screenshots can be included in the final report to demonstrate the major features of the implemented system.

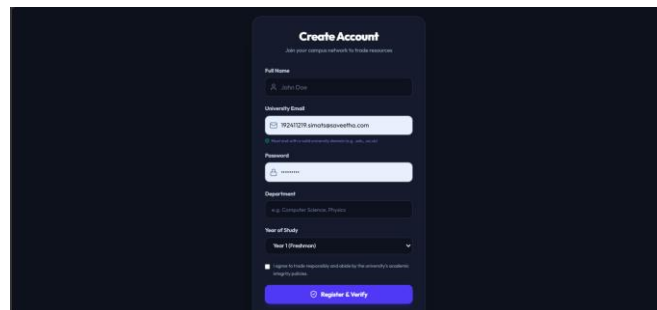


Fig 4.1 User Registration and Login Interface

The authentication interface allows users to register and securely access the ResourceHub platform. It provides the entry point for users to participate in resource-sharing activities.

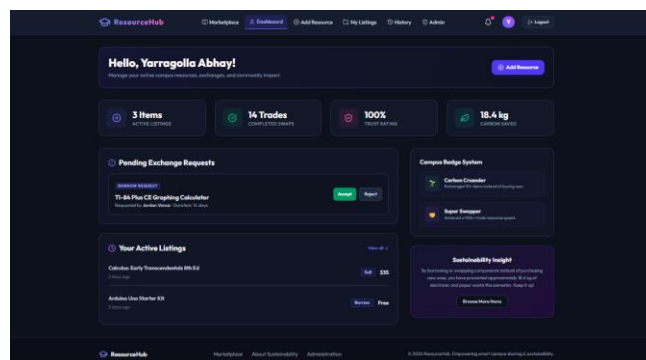


Fig 4.2 ResourceHub Dashboard

The dashboard provides users with access to available resources and the major functions of the platform. It acts as the central interface for navigating resource-related activities.

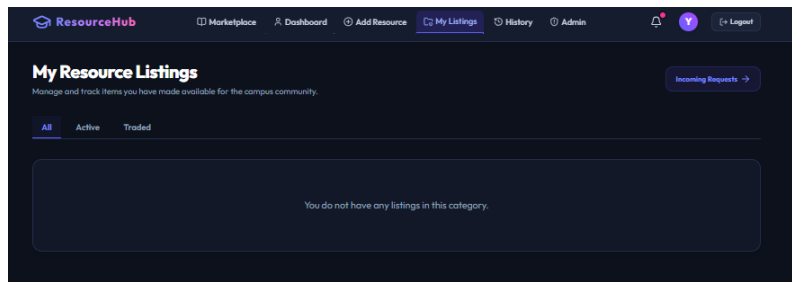


Fig 4.3 Resource Listing and Request Interface

This interface allows resource owners to publish available resources while requesters can view resource information and submit requests according to their requirements.

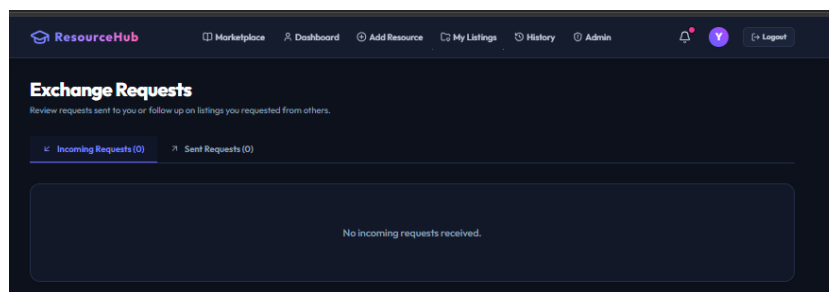


Fig 4.4 Request Management Interface

The request management interface allows resource owners to view incoming requests and accept or reject them. Requesters can also track the status of their submitted requests.

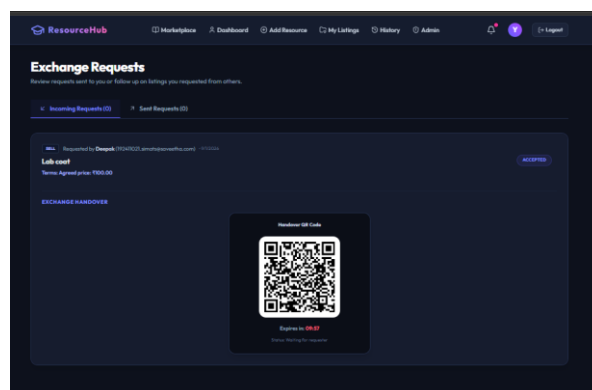


Fig 4.5 QR Handover Verification Interface

After a request is accepted, the resource owner can generate a temporary QR code for the handover. The requester can scan the QR code to verify the exchange.

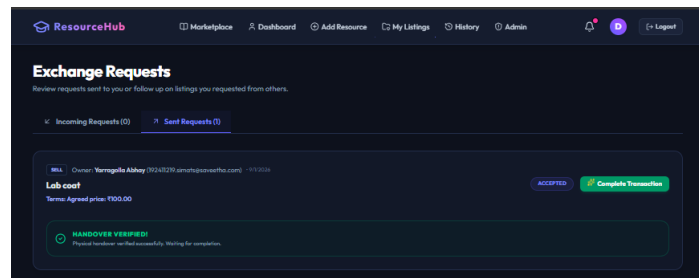


Fig 4.6 Successful Handover and Transaction Completion

After successful verification, the system updates the transaction status and marks the resource as exchanged. This confirms that the physical handover has been successfully completed.

4.2 Evaluation of Results

The ResourceHub platform was successfully implemented and evaluated using different user and transaction scenarios. The testing focused on verifying the functional correctness of the resource-sharing workflow and the security of the handover process.

The key outcomes of the project are:

- **Successful User Authentication:** Users were able to access the platform through the implemented authentication workflow.
- **Resource Management:** Resource owners were able to create and manage resource listings, while requesters could view available resources.
- **Request Processing:** Requesters could submit resource requests and owners could accept or reject them according to the transaction requirements.
- **Reservation Management:** The system maintained resource availability and transaction states to reduce conflicts during the exchange process.
- **QR Handover Verification:** The implemented QR mechanism successfully connected the accepted digital request with the physical handover process.
- **Security Validation:** The system incorporated checks for unauthorized scanning, expired verification tokens, self-scanning, and repeated verification attempts.
- **Transaction Completion:** After successful QR verification, the system correctly updated the transaction and resource states, completing the exchange workflow.

The complete workflow demonstrated that ResourceHub can provide a structured and secure mechanism for campus-level resource sharing and exchange.

4.3 Challenges Faced

Several technical and implementation challenges were encountered during the development of ResourceHub.

- **Transaction State Management:** Maintaining consistent states between resources, requests, reservations, and completed transactions required careful implementation.

Solution: A structured transaction workflow was implemented to ensure that each operation follows the appropriate state transition.

- **QR Handover Security:** A simple QR code alone would not provide sufficient security because an unauthorized or expired code could potentially be used.

Solution: Temporary verification tokens, expiry controls, authorization checks, and validation mechanisms were incorporated into the handover process.

- **Unauthorized Verification:** The system needed to ensure that only the requester associated with an accepted transaction could complete the handover.

Solution: User authorization and transaction-specific validation were applied before accepting a QR verification.

- **Self-Scanning Prevention:** A resource owner should not be able to verify their own handover by scanning the generated QR code.

Solution: The system validates the identities of the owner and requester and prevents the same user from completing both sides of the verification.

- **Repeated Verification:** A completed transaction should not be verified again.

Solution: The transaction state is checked before processing verification, preventing duplicate handover completion.

- **Integration of Multiple Modules:** Authentication, resource management, requests, reservations, and QR verification needed to operate consistently as one application.

Solution: The application was developed using modular components and tested individually before performing complete workflow testing.

4.4 Proposed Improvements

The current ResourceHub implementation can be enhanced in several ways to improve usability, security, and overall functionality:

- Implement a **rating and review system** to allow users to provide feedback after completed exchanges.
- Introduce a **trust score** based on successful transactions, ratings, and user activity.
- Add **real-time notifications** for new requests, request acceptance, QR generation, and transaction completion.
- Improve resource discovery through **advanced search, filtering, and category-based browsing**.
- Provide improved transaction history and activity tracking for users.
- Add administrative dashboards for monitoring users, resources, requests, and completed exchanges.
- Improve QR handover usability with clearer expiry indicators and verification status messages.
- Introduce additional analytics to identify frequently shared resources and overall platform usage.

These improvements can make ResourceHub more convenient for students while providing better visibility and accountability for administrators.

4.5 Future Recommendations

Future development of ResourceHub can focus on expanding the platform from a basic campus resource-sharing application into a comprehensive student resource ecosystem.

The following enhancements are recommended:

- **Trust and Reputation System:** Develop detailed user reputation profiles using ratings, reviews, successful exchanges, and transaction history.
- **Mobile Application:** Develop Android and iOS applications to provide easier access to resource listings and QR handover functionality.
- **Smart Recommendations:** Use machine learning techniques to recommend relevant resources to students based on their previous searches, requests, and usage patterns.
- **Institutional Integration:** Integrate the platform with existing university authentication or student information systems to simplify account management.
- **Advanced Notifications:** Implement email, push, or in-app notifications for important changes in requests and transactions.
- **Analytics and Reporting:** Provide dashboards showing resource utilization, popular resource categories, successful exchanges, and other system-level statistics.
- **Scalable Deployment:** Deploy the system using cloud infrastructure to support larger campus communities and multiple institutions.
- **Enhanced Security:** Introduce additional security mechanisms such as stronger session management, audit logging, and improved monitoring of suspicious activities.

In the future, ResourceHub can therefore be expanded into a scalable platform capable of supporting resource sharing across multiple departments, campuses, or educational institutions.

CHAPTER 5

REFLECTION ON LEARNING AND PERSONAL DEVELOPMENT

5.1 Key Learning Outcomes

5.1.1 Academic Knowledge

The development of ResourceHub provided a strong understanding of web-based application development, database management, authentication, transaction processing, and security concepts. The project helped in understanding how theoretical concepts from software engineering and computer science can be applied to develop a practical campus-oriented application.

The project also provided an opportunity to understand the complete lifecycle of a digital transaction, beginning with resource discovery and request submission and progressing through acceptance, reservation, handover verification, and transaction completion. Implementing QR-based verification further strengthened the understanding of secure digital transaction mechanisms.

5.1.2 Technical Skills

- **Web Application Development:** Gained practical experience in designing and developing a functional web-based platform.
- **Database Management:** Developed an understanding of storing and managing users, resources, requests, reservations, and transaction information.
- **Authentication and Authorization:** Learned how access control can be used to protect user-specific operations and transaction information.
- **QR Technology:** Gained practical knowledge of generating and validating QR-based handover verification mechanisms.
- **Backend Integration:** Improved understanding of connecting frontend interfaces with backend services and database operations.
- **Testing and Debugging:** Developed experience in identifying implementation issues, testing different workflows, and validating system behavior.

5.1.3 Problem-Solving and Critical Thinking

The development of ResourceHub involved solving several practical problems related to resource availability, request processing, transaction states, and secure handover verification. Designing a reliable QR verification workflow required careful consideration of different scenarios, including expired tokens, unauthorized users, self-scanning, and repeated verification.

Addressing these challenges improved the ability to analyze problems systematically, identify possible failure conditions, and develop appropriate solutions. The experience also strengthened debugging and decision-making skills while integrating multiple software modules into a single application.

5.2 Challenges Encountered and Overcome

Several challenges were encountered during the development and testing of the ResourceHub platform:

- Designing a consistent workflow for resource requests and reservations.
- Maintaining correct resource and transaction states throughout the exchange process.
- Implementing secure QR-based handover verification.
- Preventing unauthorized users from completing a handover.
- Preventing resource owners from verifying their own handover.
- Handling expired QR verification tokens.
- Preventing duplicate or repeated handover verification.
- Integrating multiple application modules while maintaining consistent system behavior.

Resolutions Included:

- Designing clear transaction states and validating each state transition.
- Performing repeated testing using different user and transaction scenarios.
- Implementing authorization checks before processing sensitive operations.
- Using temporary verification tokens with expiry mechanisms.
- Adding validation to prevent self-scanning and repeated verification.
- Testing individual modules before integrating the complete workflow.

- Debugging system behavior through systematic testing and error analysis.

These challenges provided practical experience in developing secure and reliable software rather than focusing only on the successful execution of the normal workflow.

5.3 Application of Engineering Standards

The development of ResourceHub incorporated software engineering principles and best practices to improve the quality and maintainability of the application.

- **Modular Design:** The system was organized into separate functional areas such as authentication, resource management, request processing, and handover verification.
- **Secure Development Practices:** Authentication, authorization, input validation, and temporary verification mechanisms were considered when implementing sensitive operations.
- **Database Integrity:** Database operations were designed to maintain consistency between resource, request, reservation, and transaction states.
- **Software Testing Practices:** Different functional and security scenarios were tested to validate system behavior.
- **Code Maintainability:** Logical separation of system components was followed to simplify debugging, maintenance, and future development.
- **Transaction Safety:** Critical operations were designed to ensure that resource and transaction states remain consistent after successful or unsuccessful operations.

The application of these principles helped improve the reliability, security, and maintainability of the ResourceHub platform.

5.4 Industry Insights

The ResourceHub project provided practical insights into how modern digital platforms manage users, resources, transactions, and security. Similar concepts are used in peer-to-peer sharing platforms, marketplace applications, reservation systems, and collaborative digital services.

One of the major insights gained from the project is that a successful software system must address not only its primary functionality but also the reliability and security of its workflows. For example, allowing users to request a resource is only one part of the problem. The system must also manage availability, authorization, reservation, physical handover, and completion.

Key industry insights gained include:

- Secure transaction workflows are essential for applications involving physical exchanges.
- User authentication and authorization are fundamental components of modern applications.
- Database consistency is important when multiple users interact with the same resources.
- Good software design must consider invalid and unexpected user actions.
- Modular architecture makes applications easier to maintain and extend.
- User experience and security must be balanced when designing digital platforms.

5.5 Conclusion of Personal Development

The development of ResourceHub significantly improved my understanding of practical software engineering and web application development. The project provided an opportunity to transform a real-world campus problem into a structured technical solution.

Through the development process, I gained practical experience in application design, database management, authentication, transaction processing, QR-based verification, testing, and debugging. Working on security-related scenarios also improved my ability to think beyond the normal execution path and consider how invalid or unauthorized actions should be handled.

The project strengthened my programming, analytical, problem-solving, and software development skills. It also increased my understanding of how different technologies and modules can be integrated to create a complete working system.

The knowledge and experience gained from ResourceHub will be valuable for future academic projects and professional software development activities. The project has provided practical exposure to designing secure, user-oriented applications for solving real-world problems.

CHAPTER 6

CONCLUSION

The project “**ResourceHub: A Secure Campus Resource Sharing and Exchange Platform with QR-Based Handover Verification**” was successfully designed, developed, and evaluated as a solution for improving resource sharing within a campus environment. The primary objective of the project was to provide a centralized platform through which students can list available resources, discover resources, submit requests, manage reservations, and securely complete physical exchanges.

The implementation of ResourceHub demonstrated how different software components can be integrated to address a real-world resource-sharing problem. The platform provides structured management of users, resources, requests, reservations, and transactions. By replacing informal communication-based resource sharing with a centralized digital workflow, the system improves resource discoverability and provides better visibility into the status of each transaction.

A major contribution of the project is the implementation of the **QR-based handover verification mechanism**. The mechanism connects the digital transaction with the actual physical exchange between the resource owner and requester. Temporary verification tokens, expiry controls, authorization checks, self-scan prevention, and protection against repeated verification provide additional security during the handover process. After successful verification, the transaction is completed and the corresponding resource status is updated.

In conclusion, ResourceHub provides a practical and structured solution for campus resource sharing and exchange. The integration of resource management, request processing, reservation control, and secure QR-based handover verification makes the platform more reliable than informal resource-sharing approaches. The project successfully addresses its primary objectives and provides a foundation for developing a larger and more feature-rich campus resource-sharing ecosystem.

REFERENCES

1. I. Sommerville, *Software Engineering*, 10th ed., Pearson Education, 2016.
2. R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed., McGraw-Hill Education, 2019.
3. OWASP Foundation, *OWASP Top 10: The Ten Most Critical Web Application Security Risks*, OWASP Foundation.
4. OWASP Foundation, *Application Security Verification Standard (ASVS)*, OWASP Foundation.
5. M. Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley Professional, 2002.
6. E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley Professional, 1994.
7. International Organization for Standardization, *ISO/IEC 25010: Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE)*.
8. International Organization for Standardization, *ISO/IEC 27001: Information Security Management Systems – Requirements*.
9. J. A. Smith and A. Kumar, "Secure QR-Based Authentication and Verification Techniques for Digital Applications," *International Journal of Computer Applications and Information Technology*.
10. Documentation and technical references for the web development framework, database system, authentication technologies, QR-code libraries, and development tools used during the implementation of ResourceHub.

APPENDICES

APPENDIX A

SOURCE CODE

A.1 User Authentication Module

This section should contain the source code responsible for:

- User registration
- User login
- Authentication
- Session/token management
- Logout functionality
- User authorization

Code:

```
const jwt = require('jsonwebtoken');

const authenticateToken = (req, res, next) => {

  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {

    return res.status(401).json({

      success: false,

      message: 'Authentication required.'

    });

  }

  const token = authHeader.split(' ')[1];

  try {
```

```
const decoded = jwt.verify(  
  token,  
  process.env.JWT_SECRET  
);  
req.user = decoded;  
next();  
} catch (error) {  
  return res.status(401).json({  
    success: false,  
    message: 'Invalid or expired token.'  
  });  
}  
};  
module.exports = authenticateToken;
```

A.2 Resource Management Module

This section contains the implementation responsible for creating, displaying, updating, and managing resource listings.

Major operations include:

- Creating a resource listing
- Viewing available resources
- Updating resource information
- Managing resource availability
- Removing or disabling listings

Code:

```
const db = require('../config/database');

const createResource = async (req, res) => {

  try {

    const {

      title,

      description,

      category_id,

      exchange_type,

      price,

      item_condition,

      meetup_location

    } = req.body;

    const owner_id = req.user.id;

    if (!title || !description || !category_id ||

      !exchange_type || !item_condition ||

      !meetup_location) {

      return res.status(400).json({

        success: false,

        message: 'Required resource fields are missing.'

      });

    }

    if (exchange_type === 'SELL' &&

      (price === undefined || price === null || price < 0)) {

      return res.status(400).json({
```

```

        success: false,

        message: 'A valid price is required for SELL resources.'

    });
}

const [result] = await db.execute(

    `INSERT INTO resources

    (owner_id, title, description, category_id,

    exchange_type, price, item_condition, meetup_location)

    VALUES (?, ?, ?, ?, ?, ?, ?, ?)`,

    [

        owner_id,

        title,

        description,

        category_id,

        exchange_type,

        price || null,

        item_condition,

        meetup_location

    ]

);

res.status(201).json({

    success: true,

    message: 'Resource created successfully.',

    resource_id: result.insertId

});

```

```

    } catch (error) {

        console.error(error);

        res.status(500).json({

            success: false,

            message: 'Failed to create resource.'

        });

    }

};

```

A.3 Request Management Module

This module manages the interaction between resource requesters and resource owners.

The major operations include:

- Submitting a resource request
- Viewing pending requests
- Accepting requests
- Rejecting requests
- Updating request status
- Maintaining the relationship between requests and resources

Code:

```

const createRequest = async (req, res) => {

    try {

        const requesterId = req.user.id;

        const {

            resource_id,

```

```

    offered_resource_id,

    borrow_duration_days
  } = req.body;

  const [resources] = await db.execute(

    `SELECT *

    FROM resources

    WHERE id = ?`,

    [resource_id]
  );

  if (resources.length === 0) {

    return res.status(404).json({

      success: false,

      message: 'Resource not found.'

    });

  }

  const resource = resources[0];

  if (resource.owner_id === requesterId) {

    return res.status(400).json({

      success: false,

      message: 'You cannot request your own resource.'

    });

  }

  if (resource.status !== 'AVAILABLE') {

    return res.status(400).json({

      success: false,

```

```

        message: 'Resource is not available.'
    });
}

const [existing] = await db.execute(
    `SELECT id
    FROM exchange_requests
    WHERE resource_id = ?
    AND requester_id = ?
    AND status = 'PENDING',
    [resource_id, requesterId]
);

if (existing.length > 0) {
    return res.status(409).json({
        success: false,
        message: 'A pending request already exists.'
    });
}

await db.execute(
    `INSERT INTO exchange_requests
    (resource_id, requester_id,
    offered_resource_id,
    borrow_duration_days,
    price_agreed)
    VALUES (?, ?, ?, ?, ?)`,
    [

```

```

        resource_id,

        requesterId,

        offered_resource_id || null,

        borrow_duration_days || null,

        resource.exchange_type === 'SELL'

        ? resource.price

        : null

    ]

);

res.status(201).json({

    success: true,

    message: 'Exchange request created successfully.'

});

} catch (error) {

    res.status(500).json({

        success: false,

        message: 'Failed to create exchange request.'

    });

}

};

```

A.4 Reservation and Transaction Module

The reservation module controls the state of resources during the exchange process.

The implementation manages states such as:

AVAILABLE → REQUESTED → ACCEPTED → RESERVED → VERIFIED → EXCHANGED

The transaction module ensures that the resource and request states remain consistent during the exchange.

Code:

```
const connection = await db.getConnection();

try {

  await connection.beginTransaction();

  const [requests] = await connection.execute(

    `SELECT *

    FROM exchange_requests

    WHERE id = ?

    FOR UPDATE`,

    [requestId]

  );

  if (requests.length === 0) {

    throw new Error('Exchange request not found.');
```

```
}
```

```
const request = requests[0];
```

```
const [resources] = await connection.execute(
```

```
`SELECT *
```

```
FROM resources
```

```
WHERE id = ?
```

```
FOR UPDATE`,
```

```
[request.resource_id]
```

```
);
```

```

const resource = resources[0];

if (resource.status !== 'AVAILABLE') {

    throw new Error('Resource is no longer available.');
```

}

```

await connection.execute(

    `UPDATE exchange_requests

    SET status = 'ACCEPTED'

    WHERE id = ?`,

    [requestId]

);

await connection.execute(

    `UPDATE resources

    SET status = 'RESERVED'

    WHERE id = ?`,

    [request.resource_id]

);

await connection.commit();
} catch (error) {

    await connection.rollback();

    throw error;
} finally {

    connection.release();
}

```

A.5 QR Handover Verification Module

The QR handover module is one of the major security components of ResourceHub.

The module performs the following operations:

- Generates a temporary handover verification token.
- Creates a QR code based on the verification information.
- Displays the QR code to the resource owner.
- Allows the requester to scan the QR code.
- Validates the token.
- Checks token expiry.
- Verifies the authorized requester.
- Prevents self-scanning.
- Prevents repeated verification.
- Updates the transaction after successful verification.

Code:

```
const crypto = require('crypto');
const generateQr = async (req, res) => {
  const transactionId = req.params.transactionId;
  const ownerId = req.user.id;
  const [rows] = await db.execute(
    `SELECT er.*, r.owner_id
    FROM exchange_requests er
    JOIN resources r
    ON er.resource_id = r.id
    WHERE er.id = ?`,
    [transactionId]
  );
  if (rows.length === 0) {
    return res.status(404).json({
      success: false,
      message: 'Transaction not found.'
    });
  }
}
```

```

    });
  }
  const transaction = rows[0];
  if (transaction.owner_id !== ownerId) {
    return res.status(403).json({
      success: false,
      message: 'Only the resource owner can generate the QR.'
    });
  }
  if (transaction.status !== 'ACCEPTED') {
    return res.status(400).json({
      success: false,
      message: 'QR can only be generated for accepted requests.'
    });
  }
  const token = crypto.randomBytes(32).toString('hex');
  const expiresAt = new Date(
    Date.now() +
    Number(process.env.QR_EXPIRY_MINUTES || 10) * 60 * 1000
  );
  await db.execute(
    `INSERT INTO qr_verifications
    (transaction_id, verification_token, expires_at)
    VALUES (?, ?, ?)`,
    [transactionId, token, expiresAt]
  );
  res.status(201).json({
    success: true,
    expires_at: expiresAt,
    verification_token: token
  });
};

```

APPENDIX B

TEST CASES

The ResourceHub system was tested using different functional and security scenarios to verify that the major workflows operate correctly.

Test Case	Input / Scenario	Expected Result	Status
Test Case 1	Valid user registration	Account created successfully	PASS
Test Case 2	Valid login credentials	User successfully logged in	PASS
Test Case 3	Invalid login credentials	Access denied with appropriate message	PASS
Test Case 4	Create a resource listing	Resource successfully created	PASS
Test Case 5	Browse available resources	Available resources displayed	PASS
Test Case 6	Submit resource request	Request successfully created	PASS
Test Case 7	Owner accepts request	Request status updated to accepted	PASS
Test Case 8	Owner rejects request	Request status updated to rejected	PASS
Test Case 9	Generate handover QR	Temporary QR code generated	PASS
Test Case 10	Authorized requester scans QR	Handover successfully verified	PASS
Test Case 11	Expired QR scanned	Verification rejected	PASS
Test Case 12	Unauthorized user scans QR	Verification rejected	PASS
Test Case 13	Owner scans own QR	Self-verification rejected	PASS
Test Case 14	QR scanned after completion	Duplicate verification rejected	PASS
Test Case 15	Successful handover	Transaction completed and resource	PASS

		exchanged	
--	--	-----------	--

We can later modify this table to exactly match the test cases you actually performed.

APPENDIX C

DATABASE AND SYSTEM DETAILS

C.1 Major Database Entities

The ResourceHub database maintains information required for users, resources, requests, reservations, and handover transactions.

Major entities include:

- **Users** – Stores user account and authentication-related information.
- **Resources** – Stores information about resources listed on the platform.
- **Requests** – Maintains requests submitted by users for available resources.
- **Reservations** – Stores reservation-related information.
- **Transactions** – Maintains the status of the resource exchange.
- **Handover Verification** – Stores or validates temporary verification information associated with the handover process.

The relationships between these entities allow the system to maintain a complete transaction lifecycle from resource listing to successful exchange.

APPENDIX D

SAMPLE INPUT DATA

The following sample data can be used to demonstrate the ResourceHub workflow.

Sample Resource

Resource Name: Data Structures Textbook

Category: Academic Book

Condition: Good

Availability: Available

Owner: Student A

Sample Request

Requester: Student B

Requested Resource: Data Structures Textbook

Request Status: Pending

After Owner Acceptance

Request Status: Accepted

Resource Status: Reserved

Handover Verification

Verification Method: QR Code

Verification Status: Pending

After Successful QR Scan

Verification Status: Verified

Request Status: Completed

Resource Status: Exchanged

This sample demonstrates the complete ResourceHub transaction lifecycle.

APPENDIX E

SYSTEM REQUIREMENTS

Hardware Requirements

The ResourceHub platform can be developed and accessed using a standard computer system.

- **Processor:** Intel Core i3 or above
- **RAM:** 4 GB minimum
- **Storage:** At least 500 MB of available space for development requirements
- **Display:** Standard monitor or laptop display
- **Camera:** Required for QR-code scanning during handover verification
- **Internet Connection:** Required when the application uses remote services or cloud-hosted components

Software Requirements

- **Operating System:** Windows 10/11 or compatible operating system
- **Development Environment:** Suitable IDE/code editor
- **Web Browser:** Modern browser such as Chrome, Edge, or Firefox
- **Database System:** Database used by the ResourceHub implementation
- **Programming Environment:** As required by the selected application stack
- **QR Code Library:** Required for QR generation and scanning
- **Version Control:** Git
- **Development Tools:** Tools required for frontend, backend, and database development

APPENDIX F

SAMPLE OUTPUT

The following represents the expected sequence of the ResourceHub handover workflow.

Resource Owner

Resource: Data Structures Textbook

Request: Accepted

[Generate Handover QR]

QR Status: Active

Expiry: Temporary

Requester

Handover Verification

[Scan QR Handover]

QR Status: Scanning...

Successful Verification

Handover Verified Successfully

Verification Status: VERIFIED

Request Status: COMPLETED

Resource Status: EXCHANGED

Invalid Verification

Handover Verification Failed

Reason:

QR token expired / Unauthorized user /

Self-scan not allowed / Transaction already completed